

Method to analyze the distribution of DCM motifs in Zebrafish Genome

2 cases:

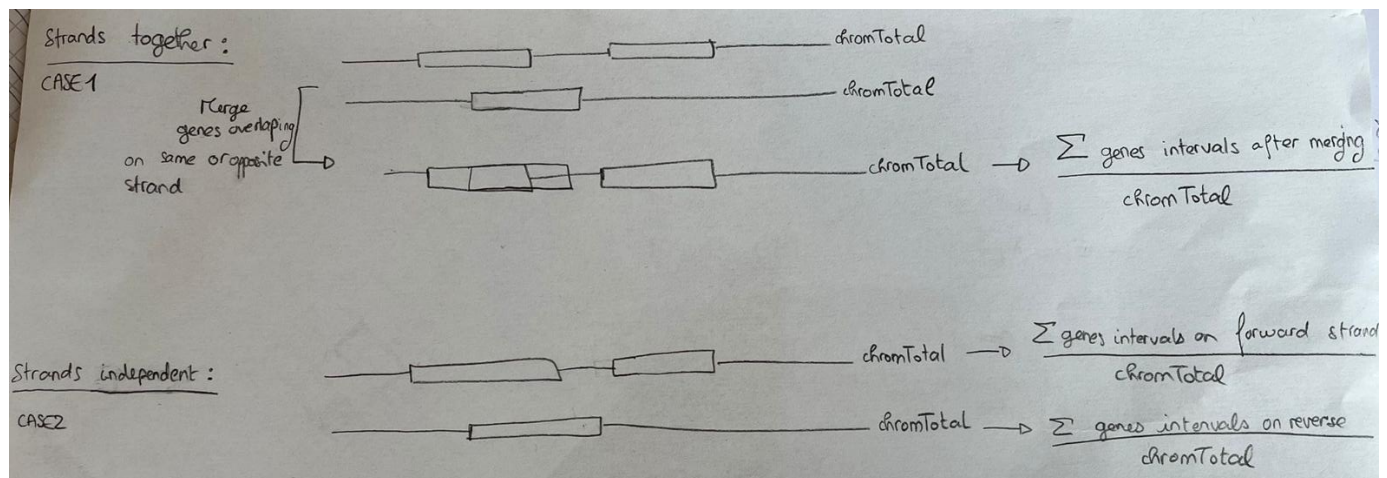
- Case 1: we merge genes overlapping on same or opposite strands
- Case 2: we merge genes only if they overlap on the same strand, i.e. consider strands independently

Where Genome Length = sum of chromosomes lengths, considering only resolved chromosomes (25) and MT chr = chromTotal which corresponds to one strand length = 1345118429

Method:

Distribution of genes in genome:

- From the genome annotation (Ensembl, GRCz11.113), we get the positions of the genes, transcripts, exons etc. And transform those intervals to PyRanges object.
- Merge genes that are adjacent or overlap to get the intervals of regions covered by genes and compute the lengths of that intervals:
 - o Case1: use gene_coverage = pyranges.merge(**strand=False**).length
 - o Case2: use gene_coverage = pyranges.merge(**strand=True**).length
- Define the intergenic region as “genome length – regions covered by genes”
- Proportion of genome covered by genes =
 - o
$$\frac{\text{Sum of genes intervals merged on both strands}}{\text{Genome Length}} [\text{case1}]$$
 - o
$$\frac{\frac{\text{Sum of genes intervals merged on forward strand}}{\text{Genome Length}} + \frac{\text{Sum of genes intervals merged on reverse strand}}{\text{Genome Length}}}{2} = \frac{(\text{Sum of genes intervals merged on forward strand}) + (\text{Sum of genes intervals on reverse strand})}{2 * \text{Genome Length}} [\text{case2}]$$



- As you go deeper in scale (to transcripts level, exons level, CDS level, UTRs) the denominator in the proportion computation is adapted.
- Granges (case1: use `reduce()` with `ignore.strand=True` ; case2: use `reduce()` with `ignore.strand=False` !! it's inverted compare to PyRanges Booleans).

Distribution of DCM motifs in genome:

- Find all motifs in genome with `fastaregexfinder` tool (<https://github.com/dariober/bioinformatics-cafe/blob/master/fastaregexfinder.py>):
"Search a fasta file for matches to a regex and return a bed file with the coordinates of the match and the matched sequence itself."
Here the sequence looked up for in the genome is `CC[A/T]GG`
➔ Output: bed file format with chr and positions of the motif, along with strand
- Convert the bed file format to PyRanges object
- Check how many motifs are contained in genes (overlap of 5 bp) using:
 - o `[case1] motifs_pyranges.join(features_genes_ranges, strandedness=False, how='containment', report_overlap=True)`

In theory, if the methylase can methylate both strands, it means that a motif that is on the opposite strand of the gene can also be methylated when that gene is being transcribed ➔ so this case (case1) should count as "motif inside gene" motifs that are to the opposite strand of the gene (`strandedness=False`).

- [case2] motifs_pyranges.join(features_genes_ranges, **strandedness='same'**, how='containment',report_overlap=True)

In this case, a motif is counted as to be “inside gene” only if it’s found on the same strand of that gene (because this case considers the methylase will only methylate motifs on the same strand of the gene).

- Subset for motifs found inside at least one gene and look if they’re found inside transcripts, exons etc

- Proportions:

- $$\frac{\text{Number of motifs found inside a gene (on same or opposite strand)}}{\text{Total number of motifs in genome}}$$
 [case1]
- $$\frac{\text{Number of motifs found inside a gene on same strand of the motif}}{\text{Total number of motifs in genome}}$$
 [case2]